

高空车项目文档

一. 项目背景和意义

随着经济的迅速发展，城市建设、公路建设及道路运输业的快速发展，人们对汽车专业化、多功能化的要求越来越高，专用汽车的需求量逐年递增，导致专用汽车的发展日新月异，并出现了专业化、轻量化、小型化、智能化、绿色节能发展趋势。其中，高空作业车是一种匠人和作业工具送到高空进行作业的专业设备，能够大大提升高空中施工人员的效率、安全性和舒适性，并降低劳动强度，它是一种用于施工作业的工程机械产品，在工程机械设备中占据着非常重要的位置，目前广泛应用于市政、风电、机场、设备安装维护等场合，其作业高度从十几米到近百米不等。随着近些年来高空车技术不断更新发展，高空车在特殊工作场景的缺陷也日渐明显，利用现有的传感器技术以及无线技术将高空系统推向智能化。

高空车有着先进的电控通讯系统，国外的高空车大多采用嵌入式控制系统，使用工程机械专用控制器实现整车的自动控制功能；CAN 总线技术广泛应用于控制系统中，同时设计出大量基于 CAN 总线的设备，包括控制器显示器等，从硬件和电气通讯两方面保证了多任务协同控制的实时通信和实时监控，提高了系统的可靠性和安全性。国内的高空车也都大多采用嵌入式系统，但在通信协议上与国外有着一点差别，物理层使用 CAN 总线技术，数据链路层使用普通串口协议，波特率为 9600，传输速率相比标准的 CAN 协议较低，现在高空车的控制系统是目前的剪叉车通过 ECU（下控制台）和 PCU（上控制台）来操作剪叉车的动作，PCU 安装在载人平台上，采集手柄信号、按键信号等发送给底盘的 ECU，由 ECU 执行车辆的行走、升降控制等。但对于目前的高空车控制系统而言，线束是整车电气的一大重要组成部分，各个电器件之间的联络全靠线束。PCU（上控制台）弹簧连接线经常出现老化、连接不稳定等问题，如果出现连接问题需要更换该弹簧连接线，造成耗材、提高了成本。PCU 线束是由车底盘延伸连接至平台的线束，主要和 PCU 进行连接通信，PCU 线束比较有讲究，对整机来说，PCU 线束是最好的线缆，在叉架上随着车的升降弯折移动，因此 PCU 线束需要耐受更高强度的弯折力，一般使用的是拖链式电缆。也正因为如此，PCU 线束经常出现线束老化、接口松动等问题，最近几年，通信行业中的各项发展机制都在不断深化改革，各行业都已经广泛应用了无线通信，一方面，无线通信具有灵

活的移动性以及扩展性，另一方面，无线通信造价的成本较低，相比于有线通信能够节约更多的资金投入，通过 PCU 直接和 ECU 无线通讯的方式节省 PCU 线束和 PCU 弹簧连接线，并提高 PCU 的寿命，可以很好的解决有线通讯带来的布线繁琐和成本问题。

本项目主要是将高空车的有线通信方式转变为无线通信，然后利用无线通信技术对高空车位置的确定，确保在无线通信的工作方式下各车辆工作互不影响，并针对此过程中可能出现的问题进行研究，其中包括通信稳定性、抗干扰能力和延时等方面的考虑。此外，我们还需要解决有线通信下存在的工况问题，例如无线配对、电子围栏和楼层确认等。

二. 项目介绍

2.1.系统需求分析

通过调研和分析，PCU 与 ECU 之间的无线通信方案需要解决以下四个关键问题：配对问题、通讯问题、供电问题和安全问题。每个问题对应的具体需求如下，(a)配对问题：PCU 与 ECU 之间应能够实现快速、稳定的配对过程，每个 PCU 设备需要有唯一的设备识别号，以便在系统中实现精确区分和管理。(b)通讯问题：无线通讯必须确保每个设备之间的连接是独立且稳定的，避免干扰。考虑到高空车的实时性要求，无线通讯应具备较低的通讯延迟，以确保操作指令的及时响应。工作环境中的信号干扰较为常见，系统应具备强大的抗干扰能力，以保证通讯的稳定性。无线通讯的信号传输速率应满足设备在高空作业中数据传输的需求。通讯距离应能够覆盖高空车的最大举升高度，确保在高空作业时设备间的通讯不受影响。(c)供电问题：在无线通讯模式下，PCU 和 ECU 必须具有独立的供电系统，无线通讯系统应具备较长的连续工作时间，以适应高空作业的需求。(d)安全问题：系统应确保操作人员在安全距离内发出指令，防止因操作不当引发安全事故。系统需具备准确的室内高度定位功能，以便在室内作业时，准确确定高空车的工作高度，提升作业安全性。经过上述问题分析以及对市面上无线遥控器的调研，表 2.1 汇总了所有需求指标的量化标准，以便于后续设计和测试阶段的实施。

表 2.1 系统需求指标量化标准表

需求分类	需求	参数	备注
配对问题	快速配对	小于 3s	
	一对一连接		
通讯问题	通讯延迟低	小于 10ms	
	抗干扰能力强		关键需求
	通信距离	40m	
供电问题	信号传输速率	大于 1.2KB	
	独立供电电源		
	连续工作时长	17 小时	
安全问题	安全操作距离	3m	关键需求
	室内高度位置	分米级	

2.2.系统总体总目标

高空车无线通信系统（以下简称本系统）旨在设计并实现一套完整的解决方案，支持 PCU 与 ECU 之间的无线通信、车辆进层楼层定位及 PCU 端电子围栏控制功能。该系统有效解决了传统有线连接（PCU-ECU）的线束约束与易损问题，克服了高空车进层后因 GPS 信号缺失导致的定位不确定性，并保障了有线系统无线化改造后 PCU 控制的安全性，为高空车的无线化与智能化发展奠定了坚实基础。

本系统通过 LORA 无线通信模块基于 PCU 与 ECU 之间原有的有线通信协议通过 LORA 模块按字节转发数据实现 PCU 与 ECU 之间的无线通信，通过控制引脚电平控制继电器模块实现车辆的急停按钮功能，通过 RC522 读卡模块和 NFC 卡实现两个设备之间的配对功能。并通过气压计和 UWB 模块采集气压数据和距离数据，经过气压高度换算以及与 UWB 采集的距离数据进行结合将车辆的控制区域分为控制区域，通信区域以及断联区域，实现高空车的安全控制。还通过加速度计和气压计实现车辆的楼层高度以及所在楼层的判断功能，如图 2.1 所示为系统的软件架构和硬件架构。

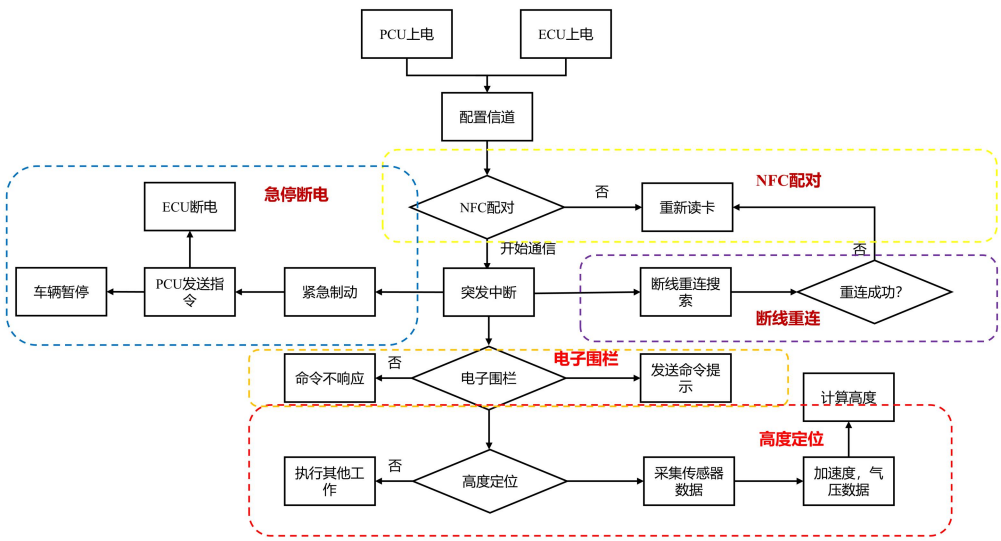


图 2.1 系统软件架构图

如图 2.1 所示，系统首先实现了 PCU 和 ECU 之间的无线通信，并设计了一套通信机制。为了避免在多辆车辆同时工作时出现通信线路拥堵，并防止设备之间的混乱，设计了车辆专属的 PCU 与 ECU 设备配对功能，每个设备都有唯一的编号，从而确保通信仅在对应的设备之间进行，避免其他设备的干扰。此外，系统还兼顾了原有产品中的急停按钮功能，以应对突发情况并实现整车断电。考虑到无线通信模块可能受到环境影响而出现中断，设计了断线重连机制。在通信中

断时，车辆将保持静止，ECU 端会发起重连请求，并设置等待时间。如果在规定时间内未能成功重连，系统将返回到配对阶段。为了提升操作安全性，系统还设计了电子围栏功能。当 PCU 离开车辆一定距离后，其发送的控制指令将不再被 ECU 执行。同时，系统还支持在室内工作时通过发送指令判断车辆的室内高度。

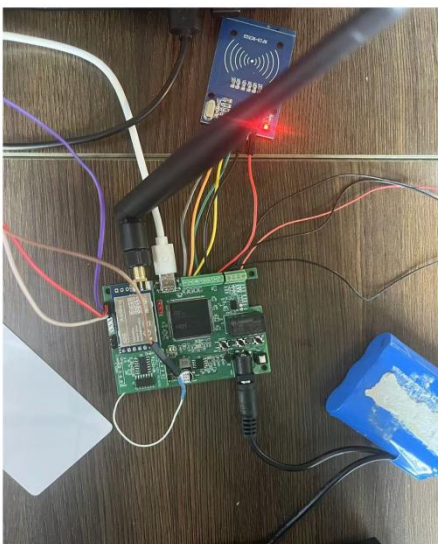
2.3 功能特性分析

2.3.1 线路连接

现有 PCU 端电路板以及 ECU 端电路板的设计如图 2.2 所示，ECU 端的电路板在设计电路板的时候留有一个 4PIN 的接线口与 ECU 端相接，PCU 端的电路板留有一个 5PIN 的接线口，接线口如表 2.2 所示。



上平台控制单元
PCU端电路板



下平台控制单元
ECU端电路板

图 2.2 PCU 与 ECU 端电路板

表 2.1 PCU 与 ECU 接线口表

设备	PCU	ECU
GND	接口 1	接口 B3
VIN1	接口 4	接口 A4
CAN_H	接口 2	接口 B1
CAN_L	接口 5	接口 C1
VIN2	接口 3	

2.3.1 通信时间分析

项目设计的无线系统是在原有的 PCU 与 ECU 的有线系统的基础上进行设计的，无线通信流程如图 2.3 所示，ECU 发送数据通过 CAN 转 RS232 后、再转 TT 发送至单片机，由单片机以同样的路径转发至 PCU 端。而 PCU 的信息以相同方式传递给单片机，再由单片机进行转发。

在实现无线通信的过程中遇到了通信延时的问题，为了实现 PCU 与 ECU 之间的正常通讯，利用示波器测试出一系列时间开销数据如表 2.3 所示。

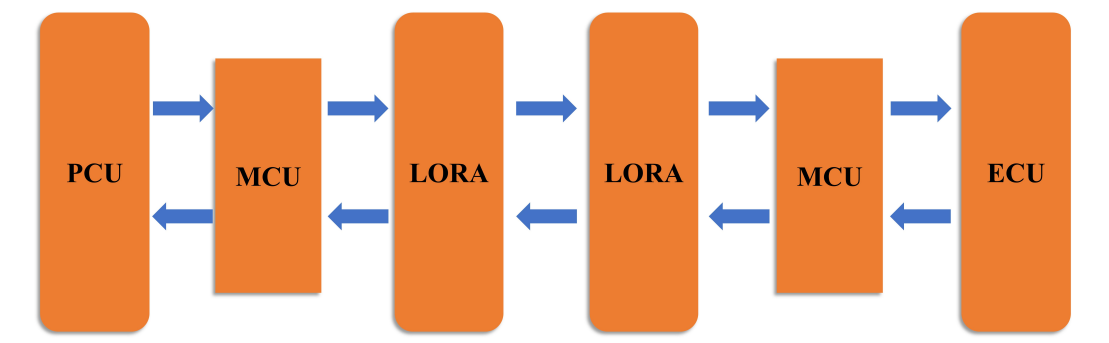


图 2.3 无线通信流程

在实现无线通信的过程中遇到了通信延时的问题，为了实现 PCU 与 ECU 之间的正常通讯，利用示波器测试出一系列时间开销数据如表 2.3 所示。

表 2.3 示波器测试时间开销数据表

数据	A0	A0-A0 01	A0 01	A0 01-1A	1A	1A-数 据	数据	数据 -1A	1A
时间	0.88	0.36	2	61,6	0.94	0.84	9.24	38.4	0.94
开销									
/ms									

在此项测试中，可以得出从 PCU 一端发送数据到单片机所连接的串口上，再由单片机转发至另一串口所连接的 ECU 的时间不到 1ms，即单片机在数据转发中消耗的时间不足 1ms，转发时间较快。

加入无线通信模块进行测试利用示波器抓取数据，结果如图 2.4 所示，在 A0 发出后 44.1ms 接收到 A0 01 数据，然而 ECU 端并没有响应，返回 PCU 数据，而是在等待 300ms 后重新发送 A0。现已知，A0 发出后会在 ECU 中有回显，需要在 10ms 内接收到 A0 01 的反馈，否则会删除 A0 重新发送。10ms 的等待时间对于无线通讯模块而言是比较苛刻的，在尝试了几种无线通讯模块后，如图 2.5 所示，数据经过无线通讯模块两次，需要开销的时间约为 40ms，单片机串口转

发开销时间约为 1ms，则总消耗时间约为 42ms。为方便今后的算法开发将通讯等待回应的时间延长至 60ms，以此来保证通讯的稳定建立，并为后续的开发留有足够的空间。

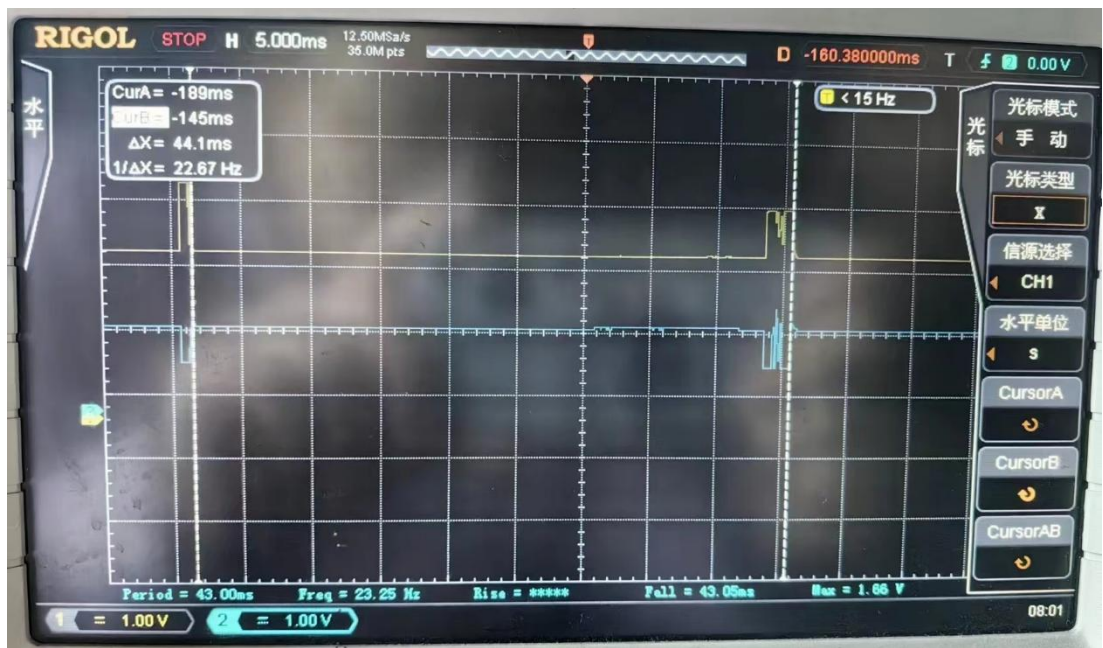


图 2.4 示波器数据抓取展示

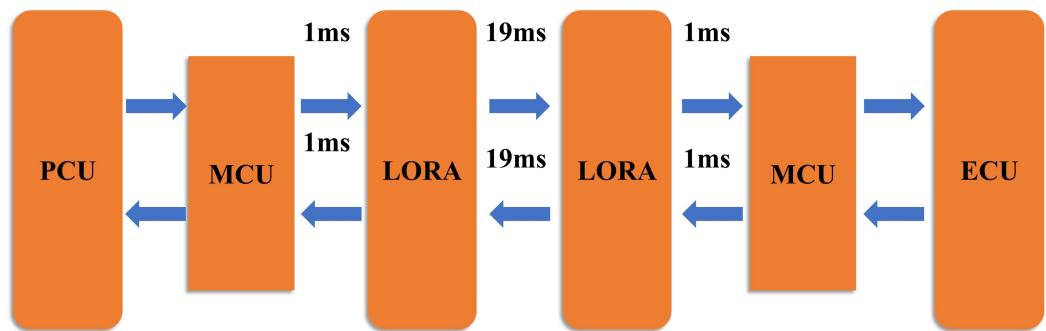


图 2.5 无线通信时间开销

2.3.2 PCU 与 ECU 端传感器介绍

PCU 和 ECU 端的传感器主要是 LORA 无线模块、加速度传感器、气压传感器、UWB 测距传感器，且 PCU 端有锂电池供电，两个设备通过无线模块进行数据传输实现系统所需功能。

配置	说明	
加速度传感器	测量通道	3 轴
	加速度分辨率	(13 位)32 mg/LSB
	加速度量程	最大为±16g

	频率范围	0.1HZ—3200HZ
	工作温度	-40℃—+85℃
气压传感器	频率范围	1Hz—200Hz
	气压量程	300hPa—1100hPa
	温度量程	-40℃—+85℃
	气压分辨率	0.01hPa
	温度精度	±1℃
	气压精度	±1hPa
LORA 无线模块	射频频率	433MHz
	通讯距离	200m
	天线	内置天线
UWB 测距模块	测距精度	30cm
	传输距离	不固定模块朝向 30m
	最大发射功率	0dBm
	测距原理	TOF
电池	容量	5000mAh
	电压范围	8.25—12.6V
	工作电流	通信中（电压 1.04v,电流 0.21A）
	连续工作时间	18.5 小时

三. 系统硬件系统

3.1 总体描述

本项目设计了一套集无线通信与室内高度定位功能于一体的智能高空车电控系统，实现了高空车的无线通讯、设备配对、电子围栏、急停断电、断电重连以及室内高度定位等功能。本章主要介绍了系统的硬件设计以及选型。根据系统需求，对 PCU 端和 ECU 端的硬件设计进行了详细规划，内容包括器件选型、最小系统设计、无线通信模块设计以及通讯延迟测试、NFC 配对电路设计、UWB 测距技术、气压计与加速度计电路设计以及电源管理模块设计。最后，通过 PCB 图设计、打板和硬件电路测试以及调试，验证系统整体方案的可行性与稳定性。

3.2 硬件电路设计

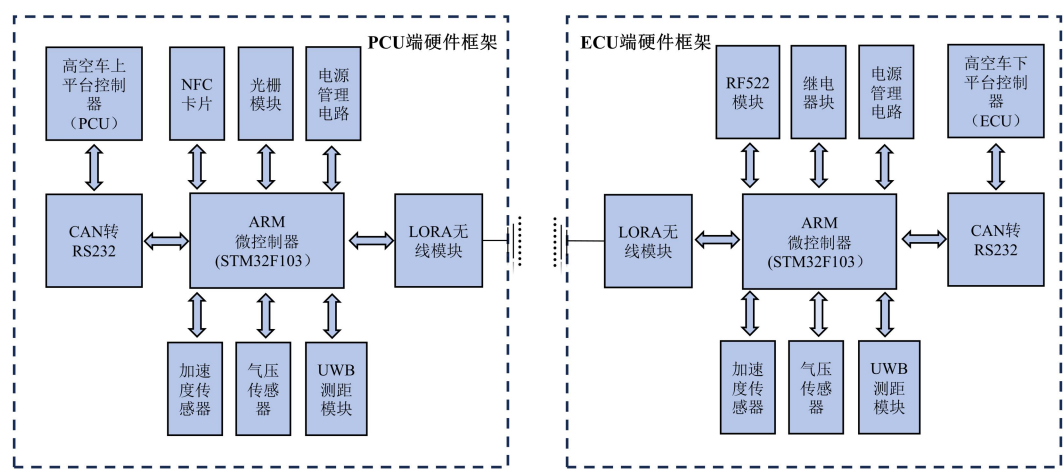


图 3.1 系统硬件架构图

如图 3.1 所示，PCU 端的硬件系统主要包括以下几个部分：ARM 微控制器、LoRa 无线通信模块、CAN 转 RS232 电路、电源管理电路、加速度传感器模块、气压传感器模块、光栅模块和 UWB 测距模块。在系统中，指令信息首先通过 CAN 转 RS232 电路从 PCU 端传输至控制器。LoRa 无线通信模块则通过 SPI 接口与 ARM 微控制器进行数据通信，将 PCU 端的指令信息以无线方式发送给 ECU 端。加速度传感器和气压传感器通过 I2C 通信协议与 ARM 微控制器进行数据交互，采集的传感器数据通过串口 1 发送至 PC 端的上位机，并进行数据保存。UWB 模块通过 UART 通信协议与微控制器通信，收集到的距离信息与气压传感器获

取的高度信息结合,实现电子围栏功能,对车辆在 180 度空间内的运动进行限制。鉴于 PCU 端原设计的供电电压为 12V,而微控制器的供电电压要求为 3.3V-5V,电源管理模块的主要功能是将 12V 电源降压至所需的 3.3V。该电源管理模块还包括充放电电路与电源保护电路,确保系统在稳定、可靠的电力供应下运行。此外,PCU 端需要独立的电源供应。光栅模块在系统中的作用主要是起到隔离的作用,将电路中的高压部分与低压部分有效隔离,从而实现高低电平的可靠读取,并对 PCU 端电路进行控制。ECU 端的硬件系统与 PCU 端有所不同,其中最显著的特点是配备了 RC522 读卡模块,主要用于实现系统设备之间的配对功能。每个 PCU 端设备都配有一张独立的 UID 卡,通过读取卡片中的 ID 信息来完成设备间的配对。此外,ECU 端还包含一个继电器模块,负责实现系统的急停功能。当 ECU 端接收到 PCU 端发送的急停指令时,继电器将根据引脚的电平信号切断电路,从而断开 ECU 端电源,切断车辆的控制电路,以确保在紧急情况下的安全停机。

3.3 模块功能介绍

上一节针对 PCU 和 ECU 端的硬件电路依据其具体功能进行细致的划分和介绍,本节主要针对 PCU 与 ECU 端硬件电路的划分模块进行详细的介绍。

3.3.1 无线通信模块

LoRa 模块凭借其长距离通信、低功耗、低延迟、易于部署、成本效益和高安全性等优点,成为物联网及其他领域无线通信的理想选择。市场上有多种 LoRa 芯片可供选择,综合考虑其经济性、低功耗、低延迟和远距离通信等特点,本系统选用为泽耀科技公司生产的 AS32-100-C 模块。

如图 3.2 所示,AS32-100-C Lora 模块是一款具有高稳定性和工业级性能的无线数据传输模块。该模块采用 LoRa 扩频调制技术,并支持 TTL 电平输出,显著提高了其抗干扰能力和稳定性。模块具有四种工作状态,并能够在不同状态之间自由切换。特别是在省电工作模式下,模块的电流消耗较低,非常适合超低功耗应用场景。



图 3.2AS32-100-C Lora 模块

下表为 AS32-100-C Lora 模块的引脚接线表：

表 3.1 Lora 模块接线表

LORA 引脚	单片机引脚
VCC	3.3V
GND	GND
MD0	GND
MD1	GND
AUX	空
RX	PB10
TX	PB11

3.3.2 RC522 读卡模块

RC522 读卡模块是基于 MFRC522 芯片的 13.56MHz 高频射频识别模块，广泛应用于门禁系统、物品管理、电子支付和图书管理等多个领域。该模块通过与 NFC 卡进行无线通信，能够实现物品或身份的识别与验证。采用 SPI 与外部微控制器进行通信。SPI 接口包括四条主要信号线：主机输出从机输入 MOSI、主机输入从机输出 MISO、时钟信号 SCK 和片选信号 SS，用于数据传输和控制信号的同步。

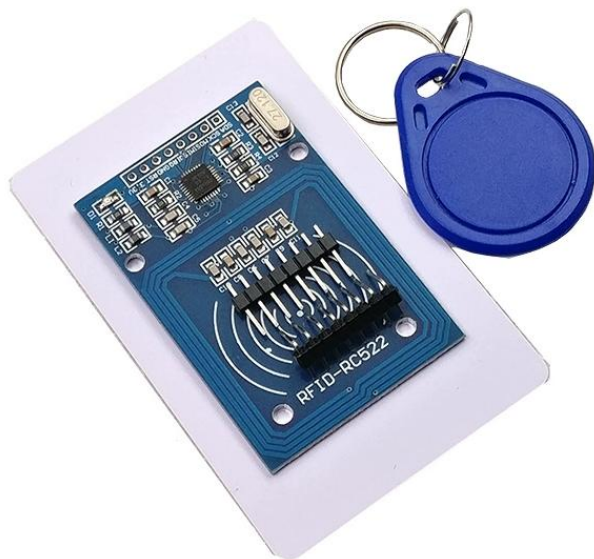


图 3.3.RC522 读卡模块

下表为 RC522 模块的引脚接线表：

表 3.2RC522 模块接线表

RC522 引脚	单片机引脚
VCC	3.3V
GND	GND
IQR	控
MOSI	PB15
MISO	PB14
SCK	PB13
SDA	PA4
RST	PB8

3.3.3 MCU 模块

微处理器采用基于 Cortex-M3 为内核的 STM32F103ZET6。该芯片拥有独立指令总线以及数据总线，运行速率最高可达 72MHz，在保持高性能的同时也兼顾了能效。除了动态运行模式外，还支持多种低功耗模式。芯片具有 32 位的数据路径、寄存器以及存储器接口，拥有高达 512K bytes 的 Flash 存储器与 64K bytes 的 SRAM，支 2 个 DMA 控制器其丰富的外设资源包括：2 个高级控制定时器、4 个通用定时器、2 个基本定时器、2 个 12 位 ADC、2 个 12 位 DAC、3 个 SPI、2 个 PC、5 个 USART、1 个 USB2.0 全速设备接口、1 个 CAN2.0B Active 接口，以及一个灵活的静态存储器控制器用于连接外部存储器。该芯片支持 -40℃至 +85℃的工作温度范围）。此外支持 SWD 和 JTAG 烧录与调试接口。

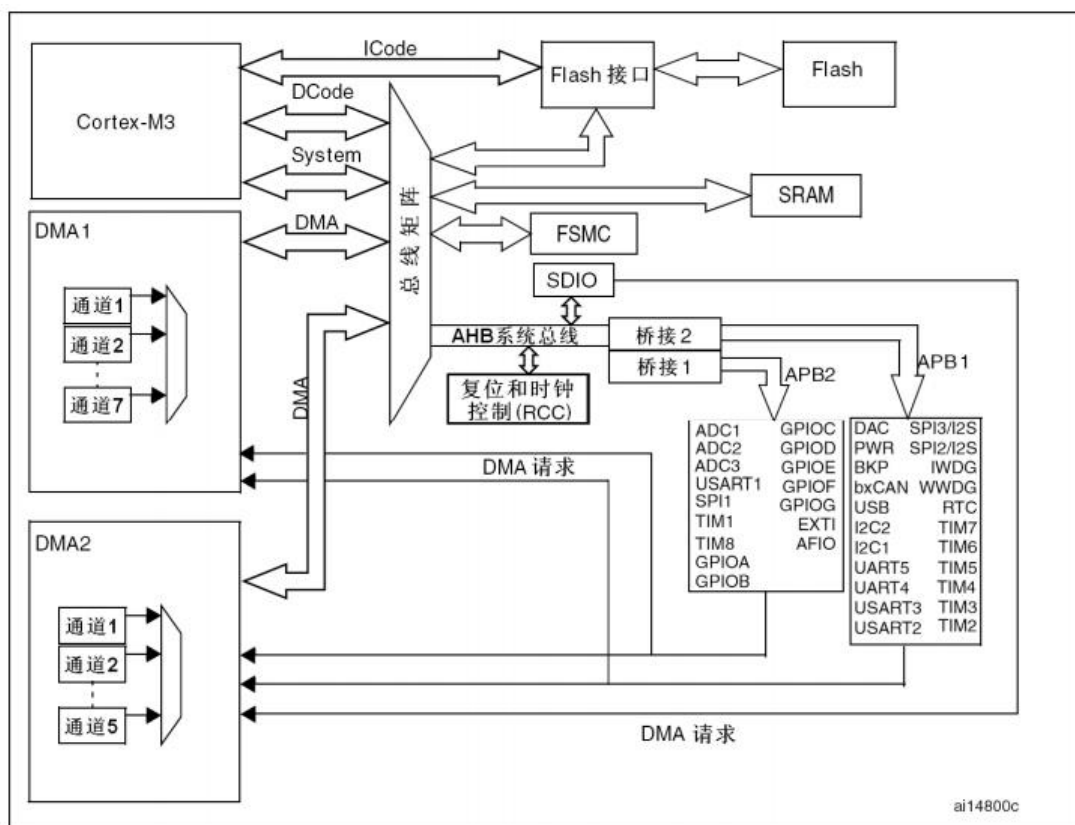


图 3.4 STM32 系统架构图

3.3.4 加速度传感器模块

ADXL345 加速度传感器是一款具有高分辨率的 3 轴数字加速度计，广泛应用于运动检测、姿态监测和重力测量等领域。在高空车的高度定位系统设计中，ADXL345 用于采集车辆在室内环境中的加速度数据，辅助实现水平运动和垂直运动的检测。该传感器具有低噪声密度、低 0g 失调漂移、超低功耗等特性，具备待机、睡眠以及工作等诸多功耗模式；分辨率最高可达 13 位，支持 $\pm 2g$ 、 $\pm 4g$ 、 $\pm 8g$ 、 $\pm 16g$ 等量程内的加速度值。

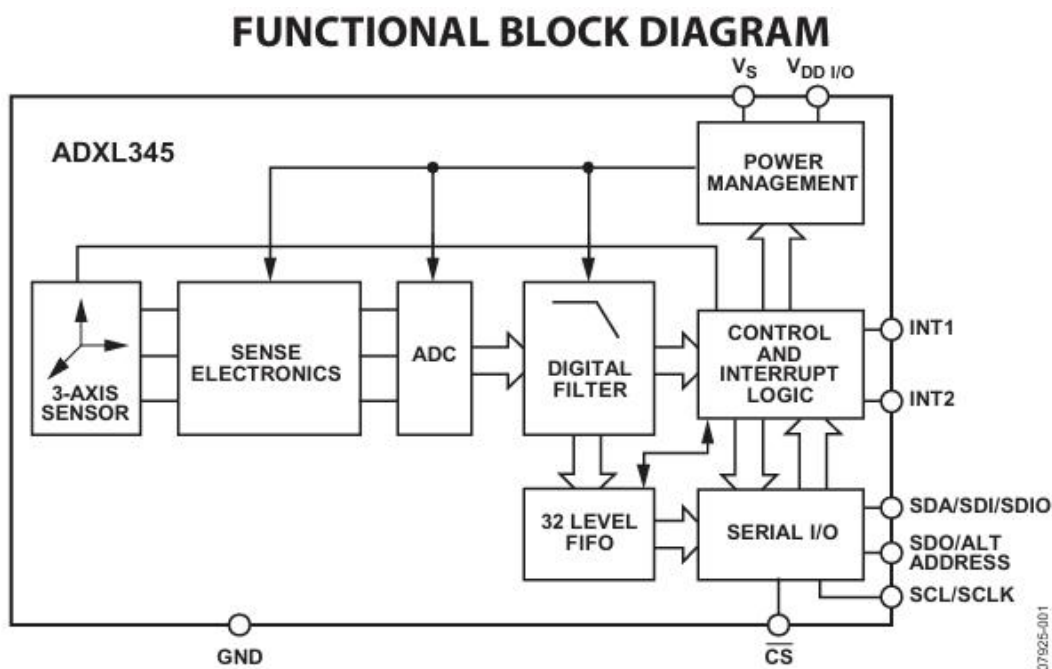


图 3.5 ADXL345 三轴加速度传感器

加速度数据采集电路如图 3.6 所示，微控制器通过 IIC 总线控制 ADXL345 进行加速度数据的采集和读取，其中 R27 和 R28 为 IIC 通信的上拉电阻，保证芯片引脚 SDA 和 SCL 开路输出的特性，确保信号的稳定。此外，ADXL345 是三轴传感器，在硬件焊接的过程中需要注意 X，Y，Z 三轴的方向，在芯片数据手册中给出了标准的方向。

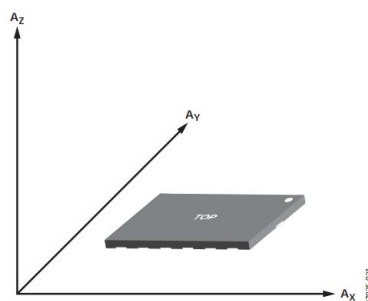


图 3.6 加速度灵敏度轴方向

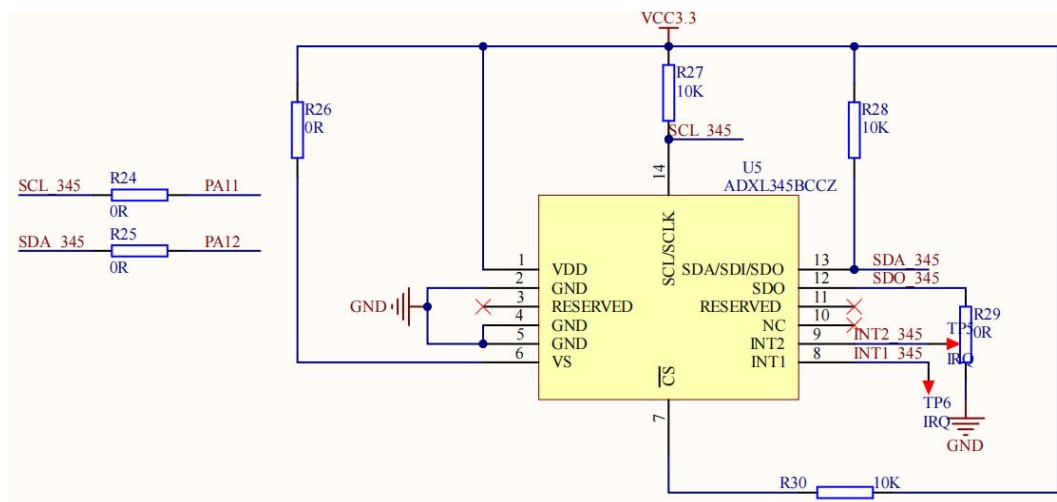


图 3.6 加速度传感器电路原理图

下表为 ADXL345 模块的引脚接线表：

表 3.3 ADXL345 接线表

ADXL345 引脚	单片机引脚
VCC	3.3V
GND	GND
SCL	PA11
SDA	PA12

3.3.5 气压传感器模块

BMP280 气压传感器是博世公司推出的一款高精度、低功耗的气压传感器，广泛应用于气象监测、高度测量、环境监控、消费电子等领域。BMP280 结合了压力和温度传感的能力，提供了精准的气压数据，并且具有极低的功耗。该传感器通过内部的压力元件单元和温度传感元件，结合数字信号处理，测量环境的气压和温度。气压测量范围可以达到 300hPa 到 1100hPa，温度测量范围可以达到 -40°C 到 +85°C，能够适应不同的环境条件。气压精度为 $\pm 1\text{hPa}$ ，能够提供 0.01hPa 的分辨率，可以检测到气压的微小变化，这对于高精度的高度测量至关重要，温度精度为 $\pm 1^\circ\text{C}$ ，支持 IIC 和 SPI 两种标准的数字接口，能够与各种微控制器进行通信。



图 3.7 BMP280 传感器

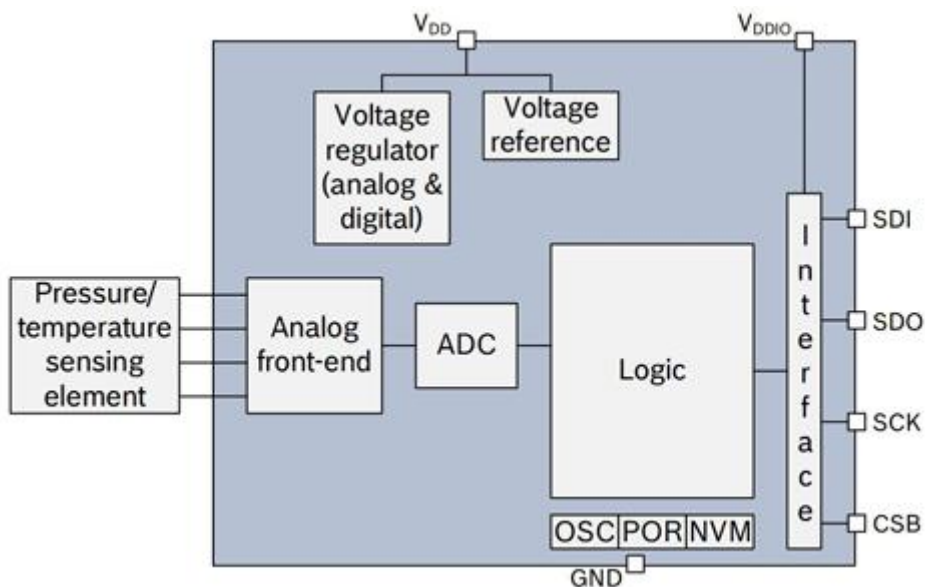


图 3.8 BMP280 芯片内部图

气压计数据采集电路如图 2.13 所示，微控制器通过 IIC 总线控制 BMP280 进行气压计数据的采集和读取，其中 R15 和 R16 为 IIC 通信的上拉电阻，芯片上的引脚 SDI 和 SCK 分别与微控制器上的 SDA 和 SCL 引脚连接，保证芯片引脚 SDA 和 SCL 开路输出的特性，确保信号的稳定。

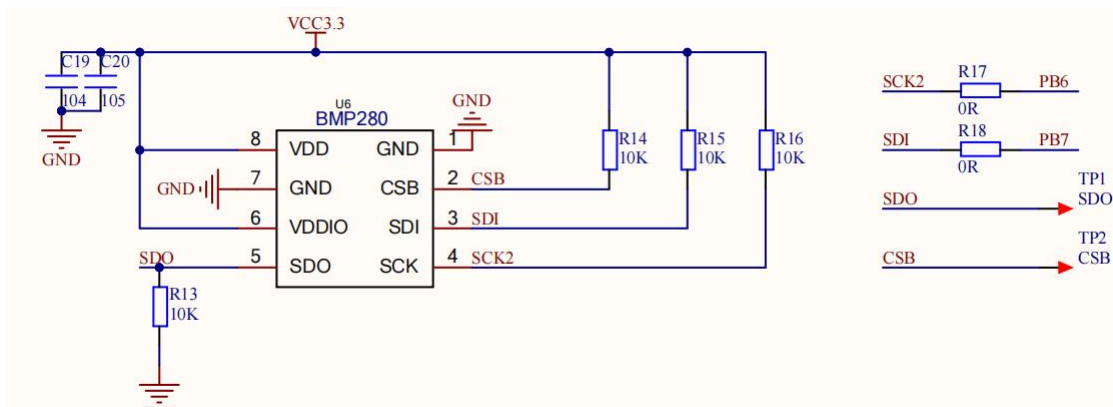


图 3.9 气压传感器电路原理图

下表为 BMP280 的模块的引脚接线表：

表 3.4 BMP280 接线表

BMP280 引脚	单片机引脚
VCC	3.3V
GND	GND
SCL	PB6
SDA	PB7

3.3.6 UWB 测距模块

UWB 测距模块如图 3.10 所示。模块采用 TTL 电平串口通信，波特率软件可配置，标准供电电压为+3.3V 并支持低功耗模式。其工业级设计确保在-40℃至 85℃环境下长时间稳定工作，超紧凑尺寸仅 15.08×26.25mm，并采用邮票孔封装设计。结合高空车的实际工作环境及其特殊的机械结构，为了提高工人操作的安全性，本系统设计了电子围栏功能。此外，为满足系统的低功耗需求，本系统选用了瀚巍微公司生产的基于 UWB 芯片 MK8000 的 MK8000 模块。该模块是一款半双工低功耗无线传输模块，内部程序开发已经完成，只需要通过串口对接即可实现数据传输。

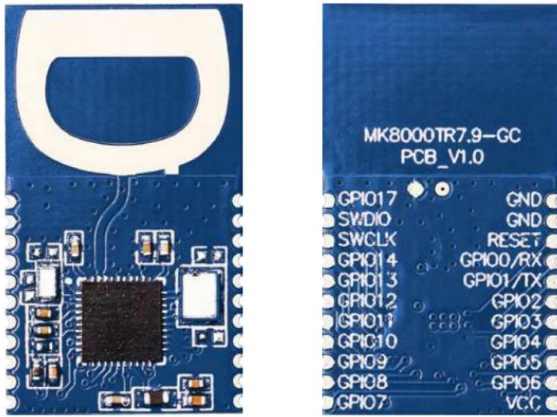


图 3.10 UWB 模块实物图

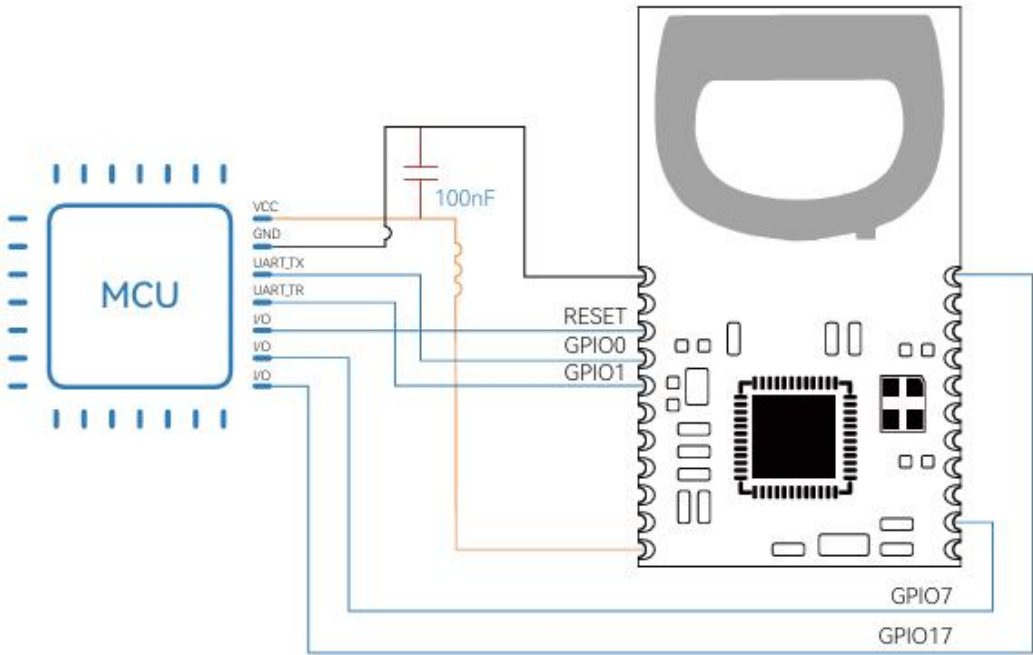


图 3.11 UWB 模块实物图

下表为 ADXL345 模块的引脚接线表：

表 3.5 BMP280 接线表

UWB 模块引脚	单片机引脚
VCC	3.3V
GND	GND
GPIO0	PC10
GPIO1	PC11

3.3.7 电源电路

在高空车原有的有线电路设计中 PCU 和 ECU 共用一个电源，现在改为无线通信之后 PCU 和 ECU 端都需要单独供电。PCU 端使用辰克公司生产的可充电锂离子电池组进行供电。该电池拥有 2800mAh 的额定容量，能够在额定条件下长时间的持久电力供应。同时，该电池具有稳定的电压输出特性，能够确保设备的正常运行。由于系统的主控芯片还有各种模块采用 3.3V 供电，PCU 采用 12V 的供电，因此，为了确保系统的各个模块能够在适当的电压下稳定工作需要设计相应的稳压电路将 12V 电压转为 3.3V。为了满足系统高效、低功耗的需求，本系统选择德州仪器生产的 TPS545DDAR 的电压转换器芯片。该芯片支持 4.5V 至 60V 的宽输入电压范围，并且输出电压可调范围从 0.8V 到 58V，内置多种保护功能，如过流保护、过压保护、过温保护和欠压锁定，从而增强系统的安全性和可靠性。

PCU 端的电压转换电路如图 3.12 所示，VIN 引脚连接外部输入电压，EN 引脚用于使能或禁用芯片，当 EN 引脚电压低于 1.2V 时，芯片关闭。输入端和输出端所接的电容都为滤波电容，输入端的电容可以用来消除输入电压的波动，输出端的电容可以用来抑制输出电压的震荡，为系统提供稳定的电压。

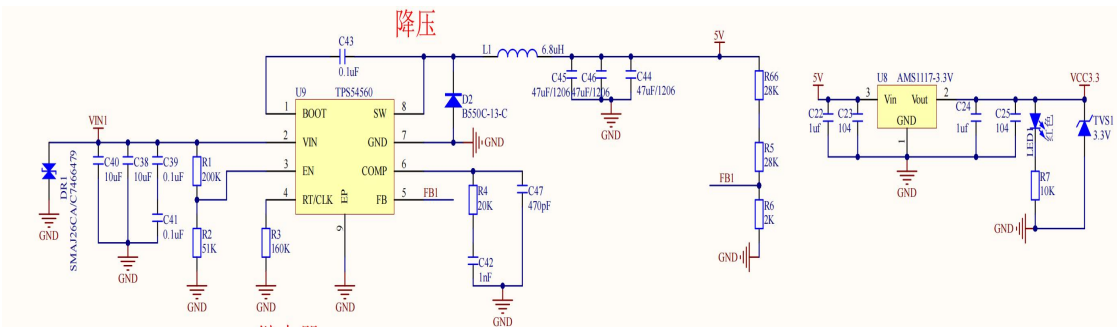


图 3.12 12V—3.3V 稳压电路原理图

3.3.8 电压隔离模块

PCU 端的供电电压为 12V，而连接 PCU 端的无限电控系统系统板供电电压为 3.3V，为了隔离高压和低压，本系统使用 EL817S1(B)(TU)-F 光电耦合器来实现电路隔离。EL817S1(B)(TU)-F 作为最新的光电隔离器，可以有效地隔离高压电路和低压电路，保护低压控制电路不受高压干扰。高低压隔离电路图如图 3.13 所示，将 12V 电源通过限流电阻 R53 接到光电耦合器的引脚 1（阳极）和引脚 2（阴极）之间的 LED 上，限流电阻的作用是防止 LED 电流过大而损坏。

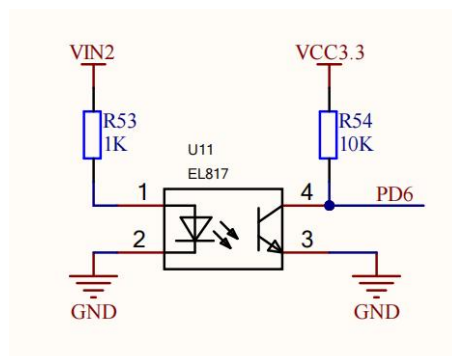


图 3.13 电压压隔离电路图

3.3.9 继电器控制电路

在原有的高空车有线系统中，设有一个急停按钮，当按下该按钮时，ECU 端会立刻断电。在将系统改为无线通信后，需要在 ECU 端增加一个继电器模块，以控制 ECU 端的供电。在按下急停按钮时，PCU 端会通过无线信号发送一个指令至 ECU 端。ECU 端接收到指令后，使得继电器所接微控制器引脚拉为高电平，从而实现电路断开。为了实现这一功能，系统选用了由雅欣电子生产的固态继电器 YX201-S-105DM。继电器内部包含一个发光二极管（LED）和一个光敏晶体管。当控制信号通过输入端时，LED 发光并产生光信号，光信号被光敏晶体管接收并转换为电流信号，从而控制输出端的负载电路。电路连接示意如图 3.15 所示。当微控制器的 PD6 引脚接收到急停指令后，该引脚拉为高电平，LED 停止发光，光敏晶体管不再导通，系统因此断电。



图 3.14 继电器实物图

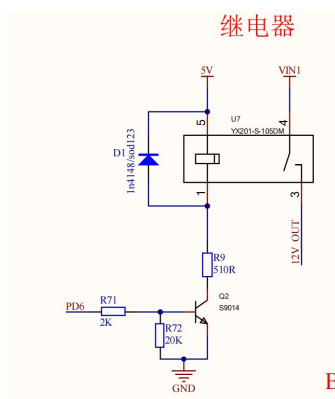


图 3.15 继电器控制电路

3.4 CAN 转 RS232 电路

高空车的 PCU 端和 ECU 端之间的信号通过 CAN 总线进行传输，如图 3.16 所示。在物理层面，CAN 信号遵循 ISO11898 标准，但从数据链路层的角度来看，实际上可以分为两种类型：第一种是标准的 CAN 通信，使用 ISO11898 标准格式，通信速率通常为 250Kbps，实际交互周期为 50ms。第二种是采用 CAN 物理层，但在数据链路层使用普通串口协议。这种方式的通信速率通常低于标准 CAN，速率由串口的波特率决定。高空车电控系统采用的是第二种模式，因此需要设计一个 CAN 转串口电路来实现数据的传输。

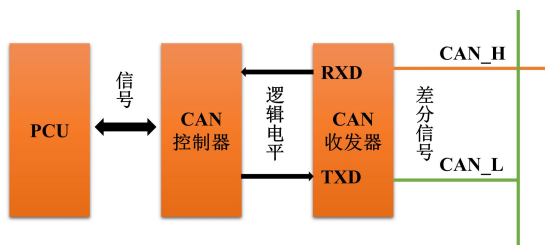


图 3.16 PCU 端 CAN 信号传输

在电路设计过程中，需要明确信号的传输方向。PCU 和 ECU 端的 CAN 信号首先通过一个 CAN 转串口电路，将 CAN 信号转换为 RS232 信号。接着，信号通过一个 RS232 转 TTL 串口电路，将其转化为 TTL 信号，传输至微控制器。最后，通过无线通信模块将数据发送出去。信号的传输流程如图 3.17 所示，CAN 转 RS232 的电路图如图 3.18 所示。

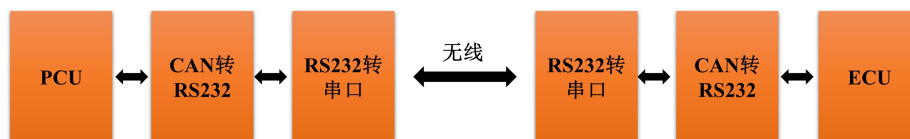


图 3.17 信号传输流程图

3.5 系统整体硬件原理图

PCU 端硬件电路如图 3.20 所示，该原理图将描述了开发 PCU 端电路板所需要的所有芯片以及原理功能单元，具体如下所示。

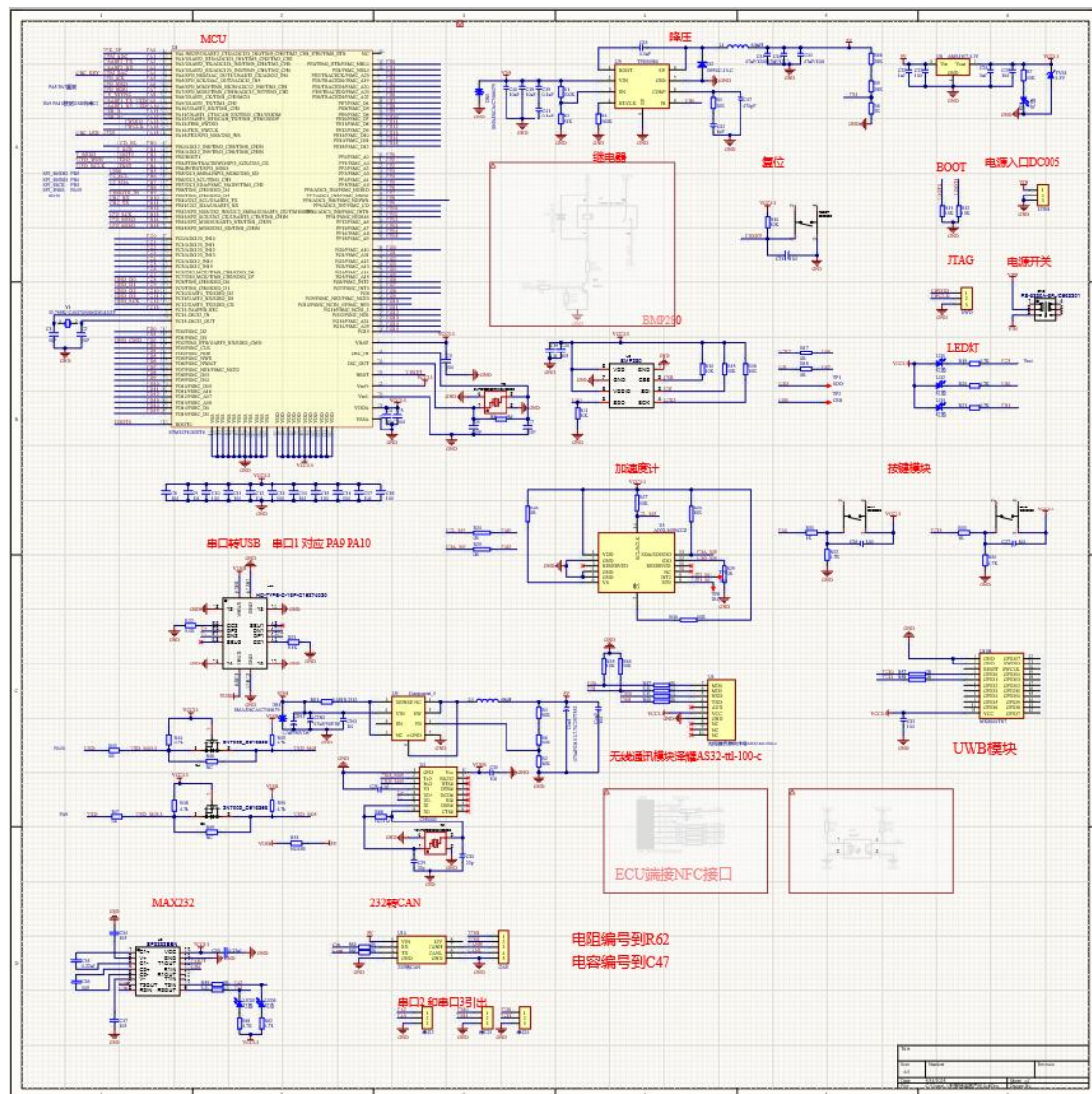


图 3.20 PCU 端硬件电路原理图

ECU 端硬件电路如图 3.21 所示，该原理图将描述了开发 ECU 端电路板所需要的所有芯片以及原理功能单元，具体如下所示。

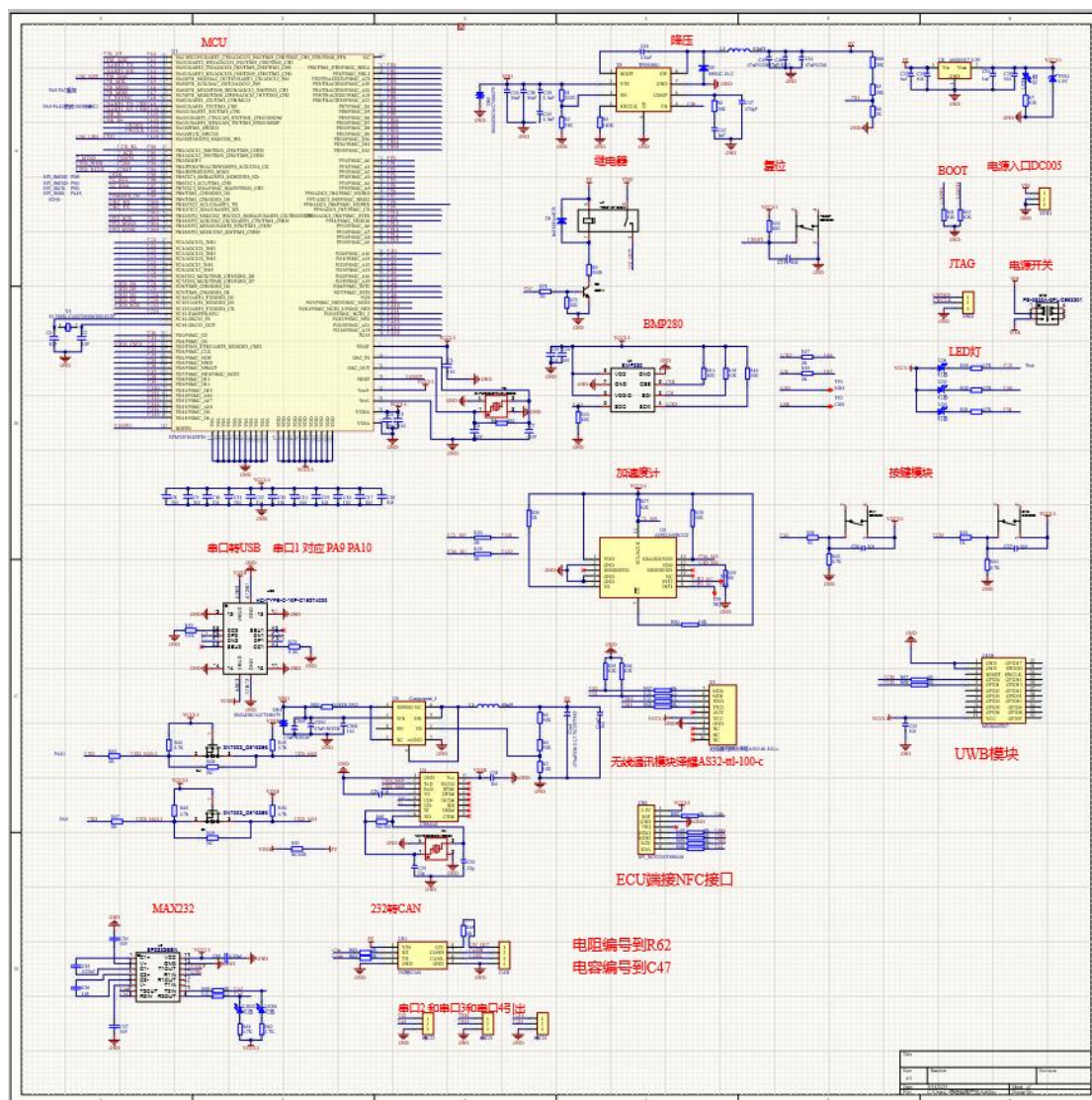
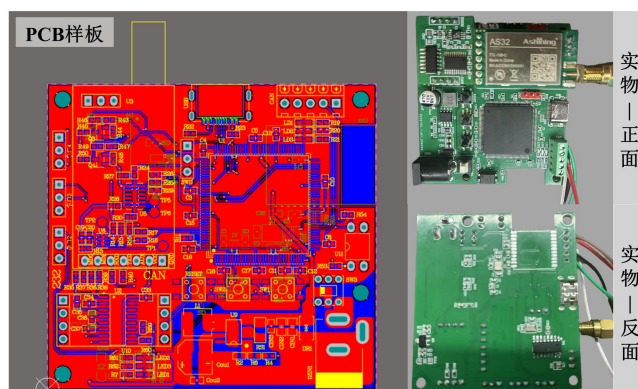


图 3.21 PCU 端硬件电路原理图

本系统使用 AltiumDesigner 软件完成原理图的设计, 并采用双层板结构进行 PCB 设计, 依据上述的 CAN 转 RS232 电路原理图, 根据外壳结构以及尺寸限制, 以及不同芯片尺寸及功能需求, 在 55mm*22mm 的范围内进行 PCB 板布局, 将所有的芯片合理规划在该范围内, 制板所得到的结果如图 3.22 所示。设计完成之后交由嘉立创公司打板制作, 对其进行调试。



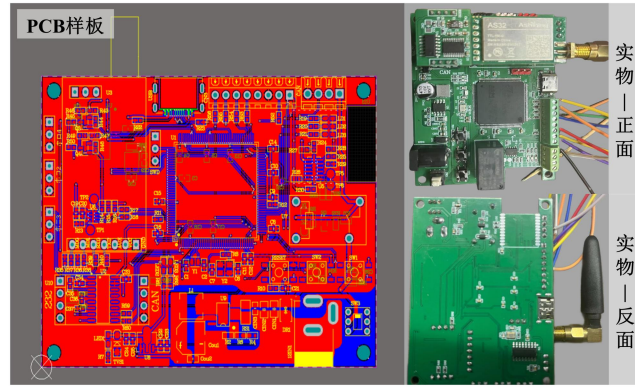


图 3.22 PCU 端实物图与 PCB 图（上）PCU 端实物图与 PCB 图（下）

四．系统软件功能逻辑设计以及代码分析

4.1 PCU 端功能逻辑设计

结合系统的实际需求、实现无线通信功能以及电子围栏和楼层高度判断的功能，功能逻辑设计主要包括有系统初始化，读取各项参数，开启各项传感器的初始化，读取数据，数据处理，数据发送，功能实现等。同时更便于现场调试与操作，该工作进行了长时间的软件测试与迭代，并最终得到如下图所示的功能逻辑流程图。

Step1. 系统初始化，包括串口、按键、指示灯、控制引脚、ADXL345 加速度传感器、BMP280 气压传感器、RC522 读卡模块、以及 UWB 模块。

Step2. 进行设备之间的配对，先发送寻卡指令，寻卡失败会再次寻卡，再进行防冲撞处理，然后选定特定卡片，读取卡号并将 4 字节二进制 UID 转换为 8 字符十六进制字符串，验证 UID 号，验证失败，重新寻卡，验证成功更新全局状态并退出。

Step3. PCU 与 ECU 配对成功，开始通信，利用无线模块转发 ECU 和 PCU 发送的数据，通信流程如下：

步骤 1. 先写串口 2 与串口 3 的中断接收回调函数，并利用全局变量置标志位

步骤 2. 判断标志位为 1 时，串口 2 接受到数据之后由串口 3 转发（无线模块），串口 3 接收到数据之后由串口 2 转发（按字节转发）

步骤 3. 判断接收到的数据，并置标志位，利用标志位实现急停按钮和断线重连的功能。

Step4. ADXL345 芯片 ID 读取，设置芯片为测量模式，配置好数据读取的参数，同时进行初始化训练，同时获取加速度数据。

Step5. BMP280 芯片 ID 读取，设置芯片为测量模式，配置好数据读取的参数，同时进行初始化训练，同时获取气压计数据。

Step6. 配置好 UWB 数据读取的参数，同时进行初始化训练，同时获取距离数据。

Step7. PCU 端利用气压数据和距离数据实现电子围栏的功能，ECU 端利用气压数据与加速度数据实现楼层判断的功能。

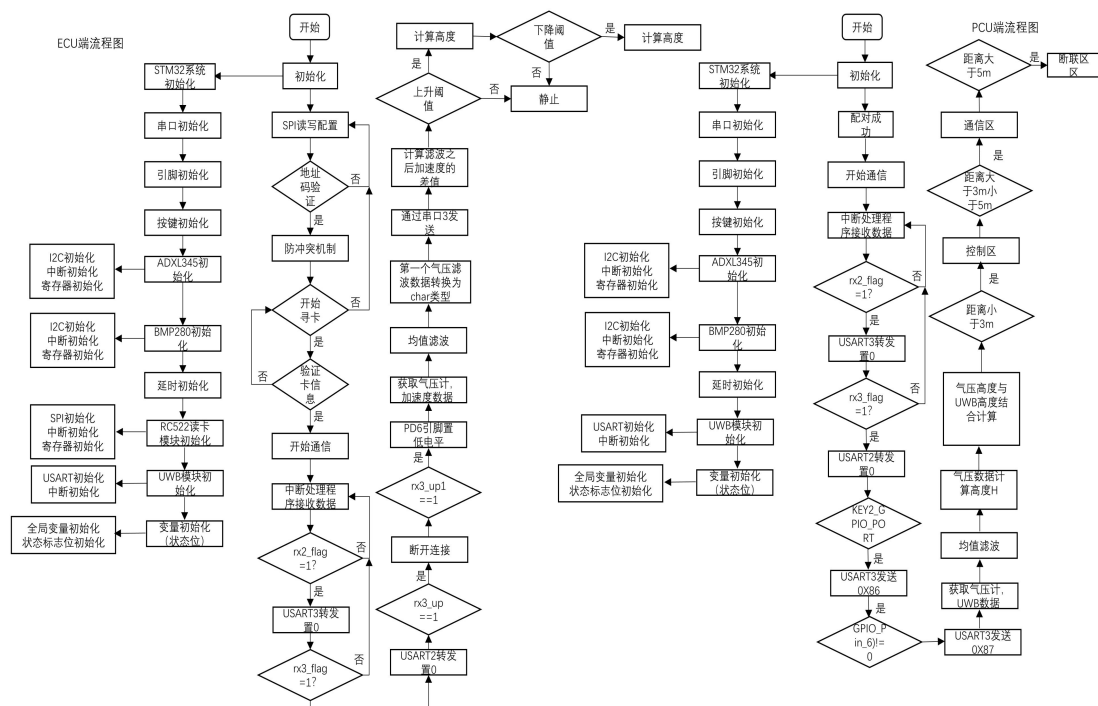


图 4.1 系统程序流程图

4.2 软件系统设计

软件系统设计主要的工作是针对 PCU 和 ECU 端的嵌入式系统的软件编程开发, 主要依据上一节中功能逻辑进行编程, 以下为主要关键环节的具体程序解释。

PCU 端系统功能初始化, 包括有系统初始化、串口初始化、延时初始化、引脚初始化、按键初始化、ADXL345 加速度传感器初始化、BMP280 气压传感器初始化、UWB 初始化以及变量初始化。

```

9 void convertCharToDouble(char* charData, double* doubleData)
10 {
11     memcpy(doubleData, charData, sizeof(double));
12 }
13
14 int main(void)
15 {
16     double BMP_Pressure, BMP_Temperature, convertedDoubleValue;
17     double base_pressure, base_Temperature, Pressure;
18     double press;
19     double H, H1, H2, L;
20     int num_sample = 50;
21     double sum = 0.0;
22     double sumnt = 0.0;
23     char buffer[sizeof(base_pressure)];
24     /*滴答时钟初始化*/
25     SysTick_Init();
26     battery_Config();
27     USART1_Config();
28     USART2_Config();
29     Lora_Usart_Config();
30     USART4_Config();
31     Key_GPIO_Config();
32     MyI2C_Init();
33     Bmp_Init();
34     printf("\r\nBMP280大气压传感器实验");
35     // Delay_us(500);
36     BMP280_ReadID();
37     printf("BMP280 ID: 0x%02x\r\n", BMP280_ReadID());
38     // BASIC_TIM_Init();
39     base_Temperature = BMP280_Get_Temperature();
40     // printf("初始基准温度值: %.3f C\r\n", base_Temperature);
41     base_pressure = BMP280_Get_Pressure();
42     // printf("初始基准气压值: %.3f Pa\r\n", base_pressure);
43 }

```

图 4.2 PCU 端程序初始化

ECU 端系统功能初始化，包括有系统初始化、串口初始化、延时初始化、引脚初始化、按键初始化、ADXL345 加速度传感器初始化、BMP280 气压传感器初始化、RC522 读卡模块、UWB 初始化以及变量初始化。

```
int main(void)
{
    double BMP_Pressure, BMP_Temperature, convertedDoubleValue;
    double base_pressure, base_Temperature;
    double press=101325;
    double H_L, H_P, H1, H2;
    float sum = 0, sumz = 0, sumt = 0;
    float average_pressure, average_temp_Z, average_Temperature;
    float previous_average_pressure = 0, previous_average_temp_Z = 0;
    float diff_pressure, diff_temp_Z;
    char buffer[sizeof(base_pressure)];
    int floor_height=3;
    int FLOOR1, FLOOR2;

    /*滴答时钟初始化*/
    SysTick_Init ();
    USART1_Config();
    USART2_Config();
    battery_Config();
    Lora_Uart_Config();
    Key_GPIO_Config();
    MyI2C_Init();
    Init_ADXL345();
    Bmp_Init();
    printf("\r\nBMP280大气压传感器实验");
    Delay_us(500);
    BMP280_ReadID();
    ADXL345_ReadID();
    printf("BMP280 ID: 0x%02x\r\n", BMP280_ReadID());
    BASIC_TIM_Init();
    /*RC522模块所需外设的初始化配置*/
    RC522_Init ();
    // printf ( "WF-RC522 Test\n" );
    /*设置工作方式*/
    PcdReset ();
    M500PcdConfigISOType ( 'A' );
    base_Temperature = BMP280_Get_Temperature();
    // printf("初始基准温度值: %.3f C\r\n", base_Temperature);
    base_pressure= BMP280_Get_Pressure();
    // printf("初始基准气压值: %.3f Pa\r\n", base_pressure);
    // convertDoubleToChar(base_pressure, buffer);
    // Uart_SendByte( DEBUG_USART3, buffer);
    // 计算高度 H
    if (press <= 0 || base_pressure <= 0) {
        // printf("压力值必须为正\r\n");
        return 1;
    }
    // 计算基准高度
    H_L=18400 * (1.0 + (base_Temperature / 273.15)) * log10(press /base_pressure);
    printf("基准高度 H: %.1f 米\r\n", H_L);
    while(1)
    {
        IC_test ();

        printf("开始通信\r\n");
    }
}
```

图 4.3 ECU 端程序初始化

这段程序实现了一个 IC 卡（如 RFID）检测系统，主要功能是：循环寻卡：持续尝试检测附近 IC 卡（PcdRequest 函数），防冲撞处理：当多张卡存在时选择一张操作（PcdAnticoll）。卡号验证：将检测到的卡 UID 转换为十六进制字符串。目标卡匹配：检查是否为特定卡号 "631F01FE"（硬编码）。成功处理：找到目标卡则休眠卡片（PcdHalt），打印成功信息，并退出循环失败处理：非目标卡则打印失败信息。

```
void IC_test ( void )
{
    char cStr [ 30 ];
    uint8_t ucArray_ID [ 4 ]; /*先后存放IC卡的类型和UID(IC卡序号)*/
    uint8_t ucStatusReturn; /*返回状态*/

    /* 按照流程操作 RC522 , 否则会导致操作失败 */
    while ( a==0 )
    {
        /*寻卡*/
        if ( ( ucStatusReturn = PcdRequest ( PICC_REQALL, ucArray_ID ) ) != MI_OK )
        {
            /*若失败再次寻卡*/
            printf ( "寻卡失败\n" );
            ucStatusReturn = PcdRequest ( PICC_REQALL, ucArray_ID ); //PICC_REQALL PICC_REQIDL
        }

        if ( ucStatusReturn == MI_OK )
        {
            //printf ( "寻卡成功\n" );
            /*防冲撞（当有多张卡进入读写器操作范围时，防冲突机制会从中选择一张进行操作）*/
            if ( PcdAnticoll ( ucArray_ID ) == MI_OK )
            {
                PcdSelect(ucArray_ID);

                sprintf ( cStr, "%02X%02X%02X%02X", ucArray_ID [0], ucArray_ID [1], ucArray_ID [2], ucArray_ID [3] );

                if ( strcmp(cStr, "631F01FE")==0 ) //92E6C251
                {
                    PcdHalt();
                    printf ( "程序运行成功%s\r\n", cStr ); //打印卡片ID632E0954
                    a=a+1;
                    rx3_up=0;
                    break;
                }
                else
                {
                    printf("程序运行失败\r\n");
                    if( Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN)==KEY_ON)
                    {
                        printf("按键被按下，重置寻卡流程。 \n");
                        ucStatusReturn = PcdRequest ( PICC_REQALL, ucArray_ID );
                    }
                }
            }
        }
        break;
    }
}
//...

```

图 4.4 设备读卡配对程序

这段程序主要是 ECU 端的串口 2 和串口 3 的中断回调函数，串口 2 的中断回调函数中并没有太多的逻辑，只是当串口 2 接收到一帧数据之后，中断标志位置 1。而串口 3 的中断回调函数是对串口 3 接收到的数据进行对比，如果串口 3 接收到的数据为 0X86 时会将标志位 rx3_up 置为 1，接收到的是 0X87 会将标志位 rx3_up1 置为 1，如果接收到的数据为 0X26 的时候会将 rx3_up1 置为 0，并将 PD6 引脚电平拉高。

```

58 void Usart_SendArray( USART_TypeDef * pUSARTx, uint8_t *array, uint16_t num)
59 {
60     uint8_t i;
61     for(i=0; i<num; i++)
62     {
63         /* 发送一个字节数据到USART */
64         Usart_SendByte(pUSARTx, array[i]);
65     }
66     /* 等待发送完成 */
67     while(USART_GetFlagStatus(pUSARTx, USART_FLAG_TC)==RESET);
68 }
69
70 void UART2_idle_callback(void)
71 {
72     if(USART_GetITStatus(DEBUG_USART2, USART_IT_RXNE)!=RESET)
73     {
74         rx2_data=USART_ReceiveData(DEBUG_USART2);
75         rx2_flag=1;
76         USART_ClearFlag(DEBUG_USART2, USART_IT_RXNE);
77     }
78 }
79
80 void UART3_idle_callback(void)
81 {
82     if(USART_GetITStatus(DEBUG_USART3, USART_IT_RXNE)!=RESET)
83     {
84         rx3_data=USART_ReceiveData(DEBUG_USART3);
85         printf("%d", rx3_data);
86         // if(rx3_data == 0x86) //接收到数据之后进行判断
87         {
88             rx3_up=1;
89         }
90         else if(rx3_data == 0x87)
91         {
92             rx3_up1=1;
93         }
94         else if(rx3_data == 0x26){
95             rx3_up1=0;
96         }
97         GPIO_SetBits(GPIOD, GPIO_Pin_6);
98         a=1;
99     }
100     else
101     {
102         rx3_flag=1;
103         rx3_up1=0;
104     }
105     // rx3_flag=1;
106     USART_ClearFlag(DEBUG_USART3, USART_IT_RXNE);
107 }
108
109
110

```

图 4.5 串口中断回调程序

这段程序是 PCU 端的通信程序，先是判断串口 2 的中断标志位 `rx2_flag` 是否为 1，如果判断为 1，串口 3 转发串口 2 接收到的数据，转发完之后将串口 2 的中断标志位置为 0，让串口 2 继续接收数据。然后判断串口 3 的中断标志位 `rx3_flag` 是否为 1，如果判断为 1，串口 2 转发串口 3 接收到的数据，转发完之后将串口 3 的中断标志位置为 0，让串口 3 继续接收数据。当按键 2 按下之后串口 3 会发送 0X86 数据，这个功能是实现主动断联，当 PD6 的引脚为高电平时，串口 3 会发送 0X87 数据，并将急停标志位 `PD6_flag` 置为 1，这个功能是实现急停按钮。当判断 PD6 为低电平且急停标志位 `PD6_flag` 为 1 时会发送 0X26，并将 `PD6_flag` 置为 1。


```

while(1)
{
    if(rx2_flag==1)
    {
        Usart_SendByte(DEBUG_USART3, rx2_data);
        rx2_flag=0;
    }
    if(rx3_flag==1)
    {
        Usart_SendByte(DEBUG_USART2, rx3_data);
        rx3_flag=0;
    }
    if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN)==KEY_ON)
    {
        Usart_SendByte(DEBUG_USART3, 0x86);
    }
    if( GPIO_ReadInputDataBit(GPIOD, GPIO_Pin_6)!=0)
    {
        Usart_SendByte(DEBUG_USART3, 0x87);
        pd6_flag=1;
    }
    else if(GPIO_ReadInputDataBit(GPIOD, GPIO_Pin_6)==0&&pd6_flag==1){
        Usart_SendByte(DEBUG_USART3, 0x26);
        pd6_flag=0;
    }
}
// Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN)==KEY_ON

```

图 4.6 PCU 端通信程序

```

while(1)
{
    /*IC卡检测*/
    IC_test ();
    // a=1;
    if(a==1)
    {
        printf("开始通信\r\n");
        if(rx2_flag==1)
        {
            Usart_SendByte(DEBUG_USART3, rx2_data);
            rx2_flag=0;
        }
        if(rx3_flag==1)
        {
            printf("%c", (char)rx3_data);
            printf("%d", rx3_data);
            Usart_SendByte(DEBUG_USART2, rx3_data);
            rx3_flag=0;
        }
        if(rx3_up==1)
        {
            printf ( "断开连接\n" );
            a=0;
            // GPIO_ResetBits(GPIOD, GPIO_Pin_6);
        }
        else if(rx3_up1==1)
        {
            GPIO_ResetBits(GPIOD, GPIO_Pin_6);
        }
    }
    if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN)==KEY_ON)
    {
        a=0;
    }
}
// a=0 (GPIO_ResetBits(GPIOD, GPIO_Pin_6);)

```

图 4.7 ECU 端通信程序

这段程序是 ECU 端的通信程序，先是判断串口 2 的中断标志位 rx2_flag 是否为 1，如果判断为 1，串口 3 转发串口 2 接收到的数据，转发完之后将串口 2 的中断标志位置为 0，让串口 2 继续接收数据。然后判断串口 3 的中断标志位 rx3_flag 是否为 1，如果判断为 1，串口 2 转发串口 3 接收到的数据，转发完之后将串口 3 的中断标志位置为 0，让串口 3 继续接收数据。当判断标志位 rx3_up 为 1 时，将 a 置 0，程序又会回到读卡配对阶段，通信断开。当判断标志位 rx3_up1 为 1 时，将 PD6 的引脚拉低。

```
// Returns pressure in Pa as double. Output value of "96386.2" equals 96386.2 Pa = 963.862 hPa
double bmp280_compensate_P_double(BMP280_S32_t adc_P)
{
    double var1, var2, p;
    var1 = ((double)t_fine/2.0) - 64000.0;
    var2 = var1 * var1 * ((double)dig_P6) / 32768.0;
    var2 = var2 + var1 * ((double)dig_P5) * 2.0;
    var2 = (var2/4.0)+(((double)dig_P4) * 65536.0);
    var1 = (((double)dig_P3) * var1 * var1 / 524288.0 + ((double)dig_P2) * var1) / 524288.0;
    var1 = (1.0 + var1 / 32768.0)*((double)dig_P1);
    if (var1 == 0.0)
    {
        return 0; // avoid exception caused by division by zero
    }
    p = 1048576.0 - (double)adc_P;
    p = (p - (var2 / 4096.0)) * 6250.0 / var1;
    var1 = ((double)dig_P9) * p * p / 2147483648.0;
    var2 = p * ((double)dig_P8) / 32768.0;
    p = p + (var1 + var2 + ((double)dig_P7)) / 16.0;
    return p;
}
#endif
double BMP280_Get_Pressure(void)
{
    uint8_t XLSb, Lsb, Msb;
    long signed Bit32;
    double pressure;
    XLSb = BMP280_Read_Byte(BMP280_PRESSURE_XLSB_REG);
    Lsb = BMP280_Read_Byte(BMP280_PRESSURE_LSB_REG);
    Msb = BMP280_Read_Byte(BMP280_PRESSURE_MSB_REG);
    Bit32 = ((long)(Msb << 12))|((long)(Lsb << 4))|(XLSb>>4); //寄存器的值,组成一个浮点数
    pressure = bmp280_compensate_P_double(Bit32);
    return pressure;
}

//温度值-℃
double BMP280_Get_Temperature(void)
{
    uint8_t XLSb, Lsb, Msb;
    long signed Bit32;
    double temperature;
    XLSb = BMP280_Read_Byte(BMP280_TEMPERATURE_XLSB_REG);
    Lsb = BMP280_Read_Byte(BMP280_TEMPERATURE_LSB_REG);
    Msb = BMP280_Read_Byte(BMP280_TEMPERATURE_MSB_REG);
    Bit32 = ((long)(Msb << 12))|((long)(Lsb << 4))|(XLSb>>4); //寄存器的值,组成一个浮点数
    temperature = bmp280_compensate_T_double(Bit32);
    return temperature;
}
/*****END OF LINE*****/
```

图 4.8 BMP280 读取气压和温度

这段程序实现了 BMP280 气压和温度传感器的数据读取与校准计算，mp280_compensate_P_double: 使用芯片校准参数（dig_P1-P9），通过多层浮点运算消除非线性误差，读取传感器 3 个寄存器（MSB/LSB/XLSB）组合成 20 位原始数据，气压和温度分别通过 BMP280_Get_Pressure 和 BMP280_Get_Temperature 获取。想要在主程序里读取气压和温度的数据直接调用这两个函数即可。

```

uint8_t ADXL345_ReadID(void)
{
    return Single_Read_ADXL345(0x00);
}
//*****

//初始化ADXL345, 根据需要请参考pdf进行修改*****
void Init_ADXL345(void)
{
    //
    Single_Write_ADXL345(0x31, 0x0B); //测量范围, 正负16g, 13位模式
    Single_Write_ADXL345(0x2C, 0x0B); //速率设定为1600hz 参考pdf13页
    Single_Write_ADXL345(0x2D, 0x08); //选择电源模式 参考pdf24页 //参考14页表7进行修改
    Single_Write_ADXL345(0x2E, 0x80); //使能 DATA_READY 中断
    // Single_Write_ADXL345(0x1d, 0x0f);
    // Single_Write_ADXL345(0x21, 0xff);
    // Single_Write_ADXL345(0x2f, 0x00);
    // Single_Write_ADXL345(0x2a, 0x07);
    // Single_Write_ADXL345(0xFD, 0x0B);
    Single_Write_ADXL345(0x1E, 0x04); //X 偏移量 根据测试传感器的状态写入pdf29页
    Single_Write_ADXL345(0x1F, 0x04); //Y 偏移量 根据测试传感器的状态写入pdf29页
    Single_Write_ADXL345(0x20, 0x00); //Z 偏移量 根据测试传感器的状态写入pdf29页
    Delay_us(500);
}
//*****
//显示x轴
void ReadData_x(void)
{
    int dis_data; //变量
    Multiple_Read_ADXL345(); //连续读出数据, 存储在BUF中

    dis_data=(BUF[1]<<8)+BUF[0];
    if (dis_data & 0x8000) {
        dis_data |= 0xFFFF0000;
    } //合成数据
    temp_X=(float)dis_data*3.9/1000+0.65; //计算数据和显示, 查考ADXL345快速入门第4页

    dis_data=(BUF[3]<<8)+BUF[2];
    if (dis_data & 0x8000) {
        dis_data |= 0xFFFF0000;
    } //合成数据
    temp_Y=(float)dis_data*3.9/1000+0.78; //计算数据和显示, 查考ADXL345快速入门第4页

    dis_data=(BUF[5]<<8)+BUF[4];
    if (dis_data & 0x8000) {
        dis_data |= 0xFFFF0000;
    } //合成数据
    temp_Z=(float)dis_data*3.9/1000-4.42; //计算数据和显示, 查考ADXL345快速入门第4页
}

```

图 4.9 ADXL345 读取加速度

这段程序实现了 ADXL345 三轴加速度传感器的驱动与数据处理, 主要包含三个核心功能, 第一个是传感器寄存器读写的验证。第二个功能是对传感器功能寄存器的配置, 量程配置 (寄存器 0x31=0x0B), 采样速率 (0x2C=0x0B), 电源模式 (0x2D=0x08), 中断设置: (0x2E=0x80), XY 轴偏移校准: (0x1E/0x1F=0x04), Z 轴补偿 (0x20=0x00) 应对重力影响。通过 ReadData_x 函数读取加速度数据, 分别读取 X,Y,Z 三轴的加速度数据。

```

for (int i = 0; i < 20; i++)
{
    BMP_Pressure = BMP280_Get_Pressure();
    BMP_Temperature = BMP280_Get_Temperature();
    ReadData_x();
    sum += BMP_Pressure;
    sumt += BMP_Temperature;
    sumz += temp_Z;
    if ((i + 1) % 20 == 0)
    { // 每20次计算一次平均值
        average_pressure = sum / 20;
        average_temp_Z = sumz / 20;
        average_Temperature = sumt / 20;

        // 计算与上次平均值的差异 &&diff_pressure < -2
        diff_pressure = average_pressure - previous_average_pressure;
        diff_temp_Z = average_temp_Z - previous_average_temp_Z;

        printf("Current Average Pressure: %.2f, Difference: %.2f\r\n", average_pressure, diff_pressure);
        printf("Current Average Temp Z: %.3f, Difference: %.2f\r\n", average_temp_Z, diff_temp_Z);
        if (diff_pressure > 2)
        {
            printf("开始下楼\n");
            H_P = 18400 * (1.0 + (average_Temperature / 273.15)) * log10(press / average_pressure);
            H1 = H_P - H_L;
            if (H1 > 0)
            {
                FLOOR1 = (H2 / floor_height) + 0.5;
                printf(" FLOOR1: %d\r\n", FLOOR1+1);
            }
        }
        if (diff_pressure < -1.5)
        {
            printf("开始上楼\n");
            H_P = 18400 * (1.0 + (average_Temperature / 273.15)) * log10(press / average_pressure);
            printf("基准高度 H: %.1f 米\n", H_P);
            H2 = H_P - H_L;
            printf("楼层高度 H: %.1f 米\n", H2);
            if (H2 > 0)
            {
                FLOOR1 = (int)(H2 / floor_height) + 1;
                FLOOR1 = (H2 / floor_height) + 0.5;
                printf(" FLOOR2: %d \r\n", FLOOR1+1);
            }
        }
        else
        {
            printf("静止\n");
            H_P = 18400 * (1.0 + (average_Temperature / 273.15)) * log10(press / average_pressure);
            H2 = H_P - H_L;
            printf("楼层高度 H: %.1f 米\n", H2);
            FLOOR1 = (int)(H2 / floor_height) + 1;
            FLOOR1 = (H2 / floor_height) + 0.5;
            printf("FLOOR2: %d \r\n", FLOOR1+1);
        }
        // 更新前一次的平均值为当前平均值
        previous_average_pressure = average_pressure;
        previous_average_temp_Z = average_temp_Z;
        // 重置求和器
        sum = 0;
        sumz = 0;
        sumt = 0;
    }
}

```

图 4.9 楼层高度判断

这段程序首先是对气压、温度以及 Z 轴加速度数据进行读取，将读取之后的数据分别进行求和，然后每 20 个数据求一次均值，计算两此求得的平均值的差异，利用气压差值和加速度差值作为判断车辆在楼层中的运动状态，当判断车辆在上升或者下降时，利用气压计进行高度计算，并通过(高度差/单层高)+0.5 实现四舍五入预处理，为后续转换为整数楼层号（通常再加 1 修正基准）提供基础。


```

    int16_t distance = 0;

    BMP_Pressure = BMP280_Get_Pressure();
    BMP_Temperature = BMP280_Get_Temperature();
    // printf("press: %.3f\t", BMP_Pressure);
    H2=18400 * (1 + BMP_Temperature / 273.15) * log10(101325 /BMP_Pressure);
    // printf("high: %.1f\r\n",H2);
    H = H2-H1;
    H=H*100;
    printf("high: %.1f\r\n",H);
    if(flag != 0)
    {
        distance |= (data[4] << 0);
        distance |= (data[5] << 8);
        printf("distance = %dcm\r\n",distance);
        double Ls=sqrt(distance*distance>H*H? distance*distance-H*H:H*H-distance*distance);
        if(Ls<50){
            printf("distance = %dcm high: %.1f\r\n",distance,H);
            // printf("distance = %dcm\r\n",distance);
            // printf("high: %.1f\r\n",H);
            // printf("您已进入不可控制区域\r\n");
        }
        else{
            // printf("您已进入不可控制区域\r\n");
            printf("您已进入不可控制区域 distance = %dcm high: %.1f\r\n",distance,H);
            // printf("high: %.1f\r\n",H);
            // printf("distance = %dcm\r\n",distance);
            // printf("high: %.1f\r\n",H);
        }
    }
    // if(flag2 != 0){

```

图 4.9 电子围栏功能判断

这段程序利用气压传感器和 UWB 测距模块实现车辆的安全控制判断，气压计算的高度距离 H 和 UWB 测距计算的水平距离 L ，计算实际斜距：当水平距离 $>$ 高度时： $Ls = \sqrt{(distance^2 - H^2)}$ ，当高度 $\geq$ 水平距离时： $Ls = \sqrt{(H^2 - distance^2)}$ 。斜距 Ls 小于 50cm 时为安全区，斜距 Ls 大于等于 50cm 为危险区。此参数只是在实验室测量的时候所设，如果要实车测试。建议将安全距离设为 3m，5m 和 8m 三个距离。

五. 系统测试

5.1 系统功耗的测试

由于高空车从原来的有线通信改为现在无线通信，PCU 端采用 2800mAh 锂电池供电，为了延长 PCU 的工作时间，避免由于供电不足影响工作，因此对系统的功耗进行测试，计算出锂电池的工作时间，工作时间为电池容量除以工作电流。利用 12V 供电电压和一个 5 Ω 电阻对高空车各个状态下的电压和干路电流进行测试。PCU 端和 ECU 端的测试结果如表 5.1 和 5.2 所示。

表 5.1 PCU 端硬件系统的功耗分析

工作状态	电压/V	干路电流/A	功率/W
未配对	0.87	0.174	2.088
通信中	1.04	0.208	2.496
急停中断	0.90	0.180	2.160
未配对（状态灯灭）	0.63	0.126	1.512
通信中（状态灯灭）	0.64	0.128	1.536

基于输入电压 12V 与电池容量 2800mAh（2.8Ah），系统在不同工作状态下的续航时间如下：通信中状态功耗最高（2.496W），续航约 13.5 小时；关闭状态指示灯可显著降低功耗，如未配对灯灭模式下功耗仅 1.512W，续航延长至 22.2 小时，本系统在典型工况下可满足单日连续作业需求，关键性能指标符合工业设备能效标准。

表 5.2 PCU 端硬件系统的功耗分析

工作状态	电压/V	干路电流/A	功率/W
未配对	0.98	0.196	2.352
通信中	0.91	0.182	2.184
急停中断	0.2	0.04	0.48
未配对（状态灯灭）	0.53	1.272	1.272
通信中（状态灯灭）	0.64	1.536	1.536

5.2 楼层高度功能的测试

通过软件程序的设计，将楼层高度估计算法部署在控制板上。为了验证该算法的性能，我们采用 ECU 端的硬件系统进行测试。实验地点选在浙江大学玉泉校区工控老楼的电梯，通过电梯从三楼下到一楼进行测试。传感器参数配置及采样频率与算法在数据采集中的参数设置保持一致。测试结果如图 5.1 所示。

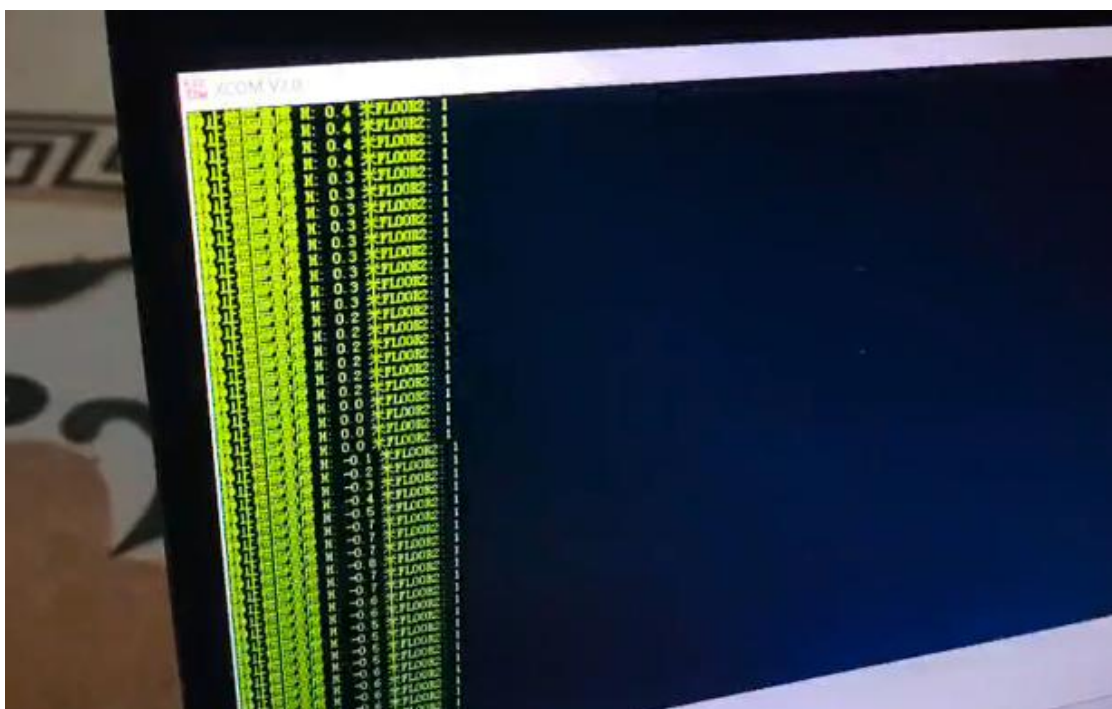


图 5.1 楼层高度功能判断

5.3 电子围栏功能的测试

系统设计完成之后，在实验室对电子围栏功能进行了测试。测试过程中，ECU 端系统通过三脚架固定并保持位置不变，而 PCU 端电路板同样固定在三角架上，并由测试人员手持，在不同高度范围内进行变化。测试过程中，通过采集高度变化时的气压值及 UWB 测距模块测得的距离，PCU 端获取相关数据后，通过无线通信将其发送至 ECU 端，再由 ECU 端上传至上位机进行数据显示，如图 5.2 所示。

press: 101688.701	distance = 91cm	press: 101693.582	distance = 40cm	high: 0.2
high: 0.6		press: 101694.338	distance = 40cm	high: 0.1
press: 101689.506	distance = 91cm	press: 101695.030	distance = 41cm	high: 0.0
high: 0.5		press: 101695.030	distance = 41cm	high: 0.0
press: 101689.506	distance = 91cm	press: 101694.256	distance = 42cm	high: 0.1
high: 0.5		press: 101693.582	distance = 42cm	high: 0.2
press: 101688.802	distance = 91cm	press: 101693.582	distance = 40cm	high: 0.2
high: 0.6		press: 101693.582	distance = 40cm	high: 0.2
press: 101689.506	distance = 91cm	press: 101693.451	distance = 43cm	high: 0.2
high: 0.5		press: 101694.154	distance = 44cm	high: 0.1
press: 101690.311	distance = 91cm	press: 101693.451	distance = 44cm	high: 0.2
high: 0.4		press: 101694.154	distance = 44cm	high: 0.1
press: 101690.311	distance = 93cm	press: 101694.969	distance = 44cm	high: 0.0
high: 0.4		press: 101694.969	distance = 44cm	high: 0.0
press: 101690.381	distance = 93cm	press: 101694.256	distance = 43cm	high: 0.1
high: 0.4		press: 101695.030	distance = 42cm	high: 0.0
press: 101687.568	distance = 95cm	press: 101694.256	distance = 42cm	high: 0.1
high: 0.7		press: 101693.582	distance = 43cm	high: 0.2
press: 101689.678	distance = 95cm	press: 101692.849	distance = 43cm	high: 0.2
high: 0.5		press: 101692.920	distance = 45cm	high: 0.2
press: 101689.678	distance = 96cm	press: 101692.920	distance = 45cm	high: 0.2
high: 0.5		press: 101692.920	distance = 44cm	high: 0.2
press: 101689.076	distance = 96cm	press: 101692.920	distance = 44cm	high: 0.2
high: 0.6		press: 101692.920	distance = 42cm	high: 0.2
press: 101689.780	distance = 94cm	press: 101692.920	distance = 42cm	high: 0.2
high: 0.5		press: 101692.217	distance = 41cm	high: 0.3
press: 101689.780	distance = 94cm	press: 101692.920	distance = 43cm	high: 0.2
high: 0.5		press: 101693.725	distance = 43cm	high: 0.2
press: 101689.780	distance = 96cm	press: 101693.022	distance = 40cm	high: 0.2
high: 0.5		press: 101692.319	distance = 40cm	high: 0.3
press: 101689.881	distance = 77cm	press: 101692.420	distance = 47cm	high: 0.3
high: 0.4		press: 101692.420	distance = 47cm	high: 0.3
press: 101689.881	distance = 77cm	press: 101693.123	distance = 60cm	high: 0.2
high: 0.5		press: 101693.123	distance = 60cm	high: 0.2
press: 101689.881	distance = 77cm	press: 101691.615	distance = 68cm	high: 0.3
high: 0.5		press: 101691.615	distance = 68cm	high: 0.3
press: 101689.881	distance = 77cm	press: 101691.615	distance = 71cm	high: 0.3
high: 0.5				

图 5.2 电子围栏功能判断

5.4 无线通信测试

如图 5.3 所示，本次实验选取了浙江大学玉泉校区的智泉路作为实验地点。实验中，使用卷尺测量每隔 50 米设置一个测试点，总测试长度为 200 米。实验主要测试了 LoRa 无线通信在不同距离下的丢包率。此外，由于校园内存在树木等障碍物，它们可能对无线信号产生一定的衰减和干扰，因此也对实验结果产生了影响。

表 5.3 无线通信测试

距离(m)	发送数据个数	接收数据个数	丢包率
50	50	50	0%
100	50	50	0%
150	50	50	0%
200	50	49	1%
250	50	43	12.5%

该数据表明，在 150m 以内通信较稳定，250m 时丢包率显著上升，影响通信质量。对于高空车这种短距离通信来说 150m 的通信距离已经完全可以实现需求。